

API INTEGRATION & DEVTOOLS NETWORK ANALYSIS REPORT

Smart Expense Tracker Web Application

Prepared By: Raghav Marda

Internship: Cyber Security Internship – Future Interns

Assessment Focus: API Integration & Network Analysis

Assessment Date: 20 May 2026

Table of Contents

• 1. Executive Summary.....	3
• 2. Introduction.....	3
• 3. Objectives.....	3
• 4. Tools & Technologies Used.....	3
• 5. Scope of Analysis.....	4
• 6. Methodology.....	4
• 7. Application Login Interface.....	5
• 8. Dashboard Initial State.....	6
• 9. Network Requests Overview.....	7
• 10. Fetch/XHR Requests Analysis.....	8
• 11. Monthly Summary API Header Inspection.....	9
• 12. Empty JSON Response Analysis.....	10
• 13. Add Expense API Payload Analysis.....	11
• 14. Add Expense API Response Analysis.....	12
• 15. Detailed Payload Structure Analysis.....	13
• 16. Detailed Request Header & Session Analysis.....	14
• 17. Security Observations.....	15
• 18. Conclusion.....	16

1. Executive Summary

This report presents the analysis of API communication and browser-based network monitoring performed on the Smart Expense Tracker web application using Chrome Developer Tools.

The assessment focused on Fetch/XHR monitoring, payload analysis, response validation, session inspection, and frontend-backend interaction analysis. Multiple API communication workflows were examined to better understand request handling mechanisms, authenticated communication behavior, and browser-based debugging practices commonly used during modern web application development and security analysis.

2. Introduction

Modern web applications heavily depend on APIs to establish communication between frontend interfaces and backend services. APIs enable applications to exchange data dynamically, process user requests, and update information in real time.

Browser-based debugging and network monitoring tools play a critical role in analyzing application behavior, troubleshooting communication issues, and validating request-response workflows. Chrome Developer Tools provides developers and security analysts with visibility into API traffic, session handling, request headers, and JSON payload structures.

Monitoring API communication is also important from a security perspective because it helps identify improper request handling, insecure data exposure, authentication issues, and inefficient communication patterns. This assessment focused on understanding practical frontend-backend interaction workflows through browser-side network inspection and DevTools analysis.

3. Objectives

- The objectives of this assessment were to understand API communication workflows within modern web applications and analyze frontend-backend interaction mechanisms using Chrome DevTools.
- The assessment also focused on inspecting Fetch/XHR requests, monitoring request-response communication, analyzing JSON payload structures, observing session handling behavior, and understanding browser-based debugging workflows commonly used during web application development and security analysis.
- Monitor payloads and responses
- Evaluate request headers
- Document security observations

4. Tools & Technologies Used

- Chrome Developer Tools
- Flask
- SQLite
- Chart.js
- JavaScript
- Render Deployment

Professional Tools & Technologies Reference

Tool	Purpose
Chrome Developer Tools	API Monitoring & Debugging
Flask	Backend Framework
SQLite	Database Management
Chart.js	Analytics Visualization
JavaScript	Frontend Interaction
Render Deployment	Cloud Hosting

5. Scope of Analysis

The scope of this analysis included browser-side inspection of API communication within the Smart Expense Tracker application. The assessment focused on monitoring authenticated API requests, analyzing request-response structures, observing session handling behavior, and inspecting JSON payload transmission.

Frontend analytics interaction, expense-management workflows, dashboard updates, and API-driven data rendering mechanisms were also analyzed during the assessment. The analysis remained limited to observable browser-side behavior and network communication accessible through Chrome Developer Tools.

6. Methodology

1. User authentication testing and session initialization
2. Dashboard interaction and analytics loading observation
3. Network request monitoring using Chrome DevTools
4. Fetch/XHR filtering for API traffic isolation
5. Request header and response header inspection
6. JSON payload structure analysis

API response validation and data rendering observation

7. Application Login Interface

This screenshot shows the login interface of the Smart Expense Tracker application before authentication. The login system is responsible for validating user credentials and initiating authenticated user sessions. This stage represents the starting point of API communication between the frontend and backend server.

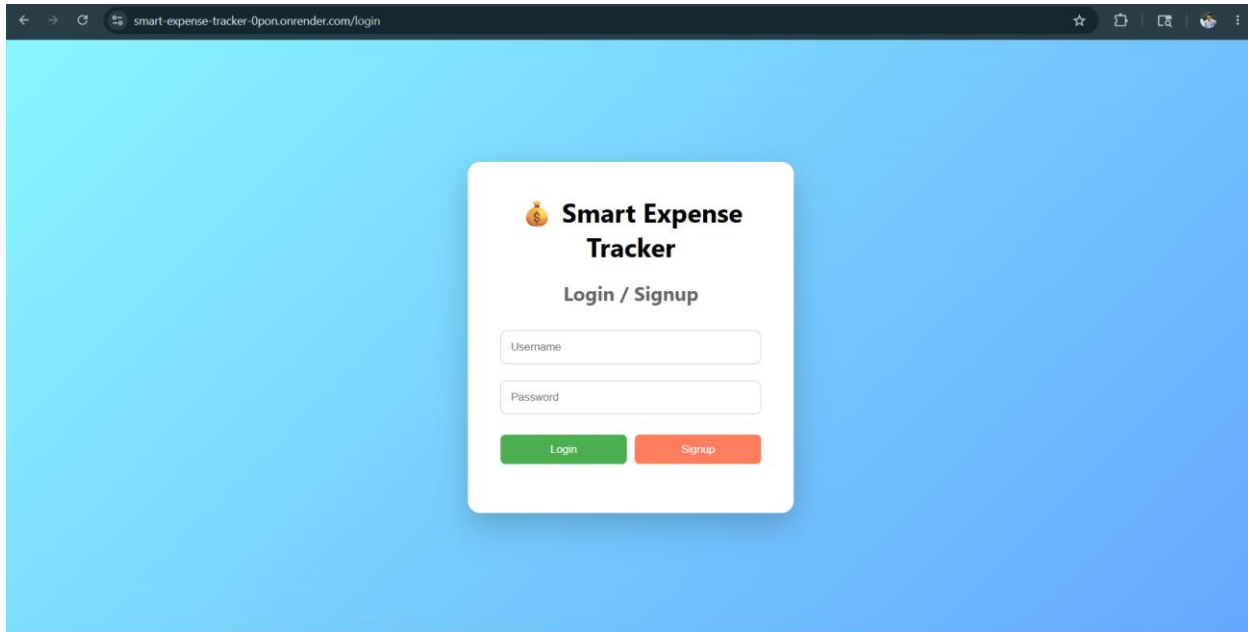


Figure 1: Smart Expense Tracker login interface.

8. Dashboard Initial State

This screenshot displays the dashboard after successful authentication. The dashboard loads user-specific financial analytics and expense information through backend API communication. Successful rendering confirms proper frontend-backend interaction.

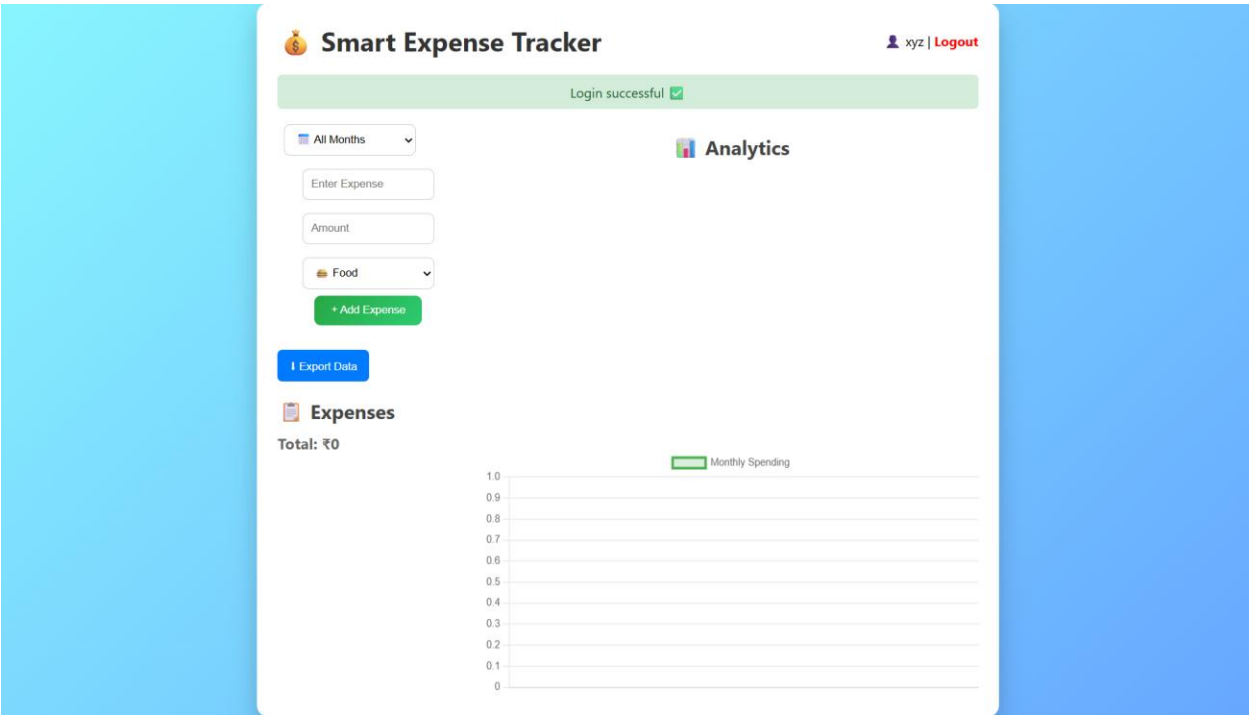


Figure 2: Dashboard state after successful authentication.

9. Network Requests Overview

This screenshot demonstrates the Network monitoring overview within Chrome DevTools. Multiple API requests can be observed during dashboard interaction, including analytics retrieval and expense-fetch operations.

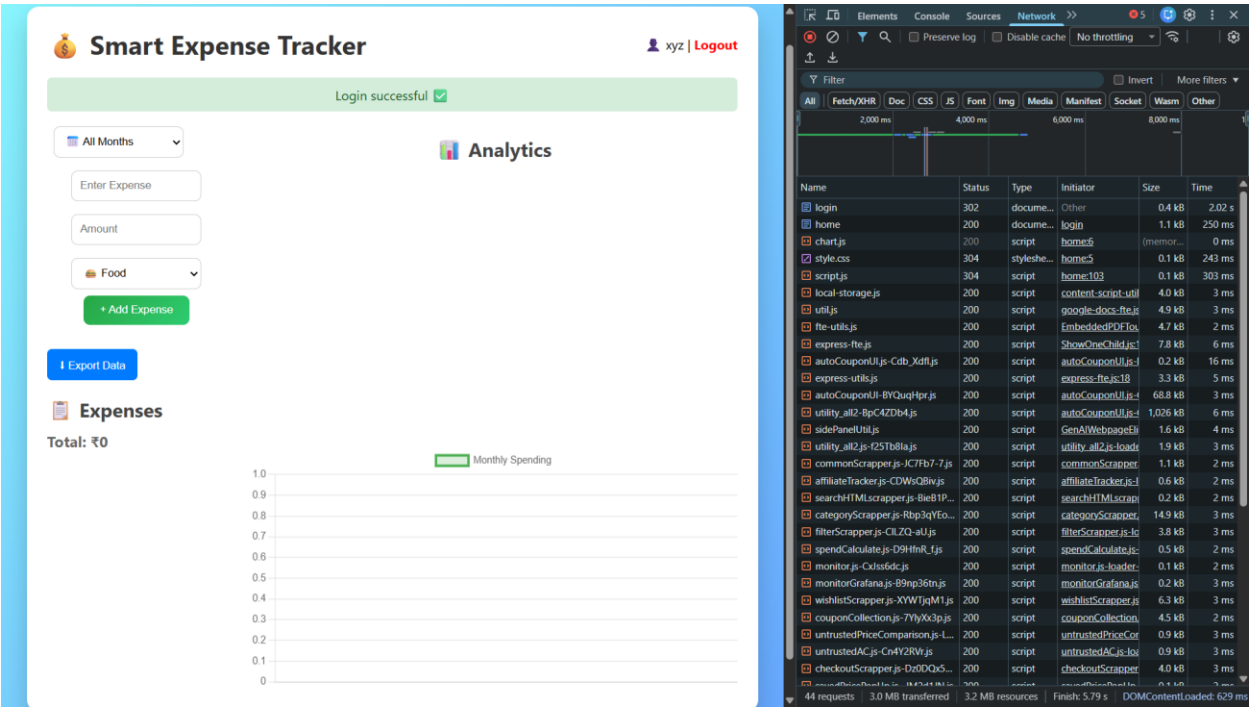


Figure 3: Overall network request overview.

10. Fetch/XHR Requests Analysis

The Fetch/XHR filter was used to isolate API communication requests from other browser resources. This helped in analyzing backend communication workflows more effectively.

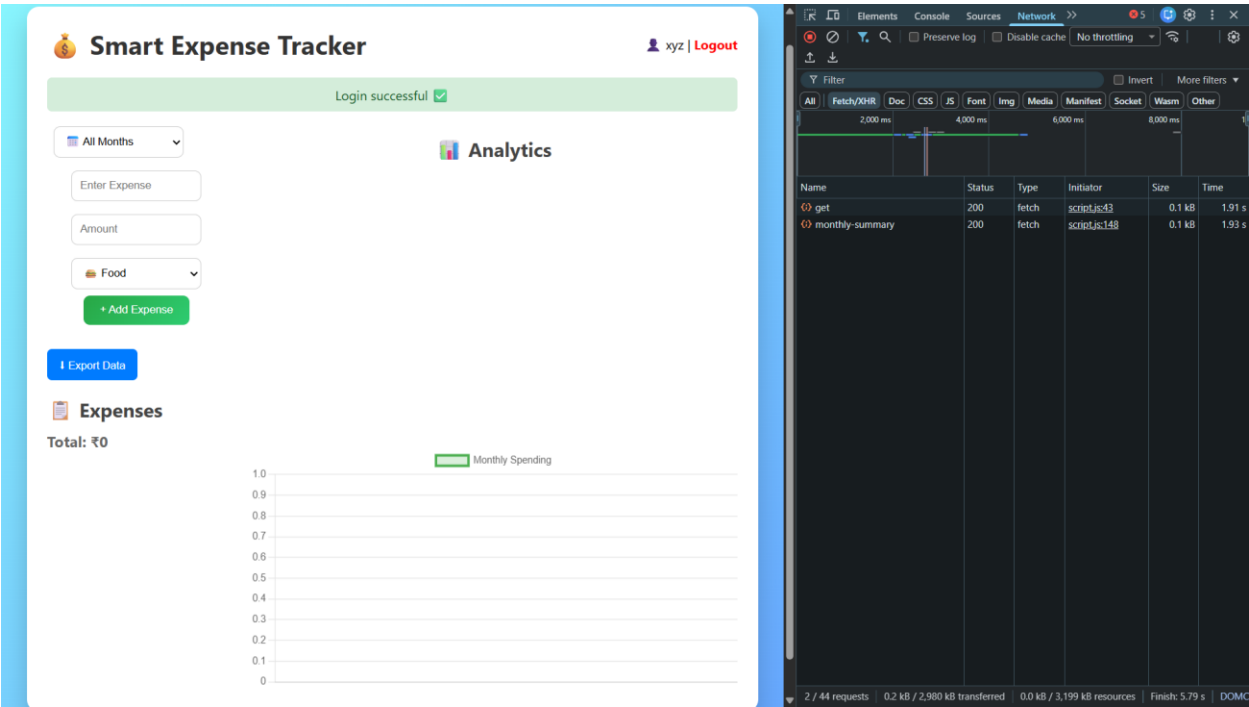


Figure 4: Fetch/XHR API communication analysis.

11. Monthly Summary API Header Inspection

This screenshot displays the request headers of the monthly-summary API endpoint. HTTPS communication and successful HTTP 200 responses were observed during the analysis.

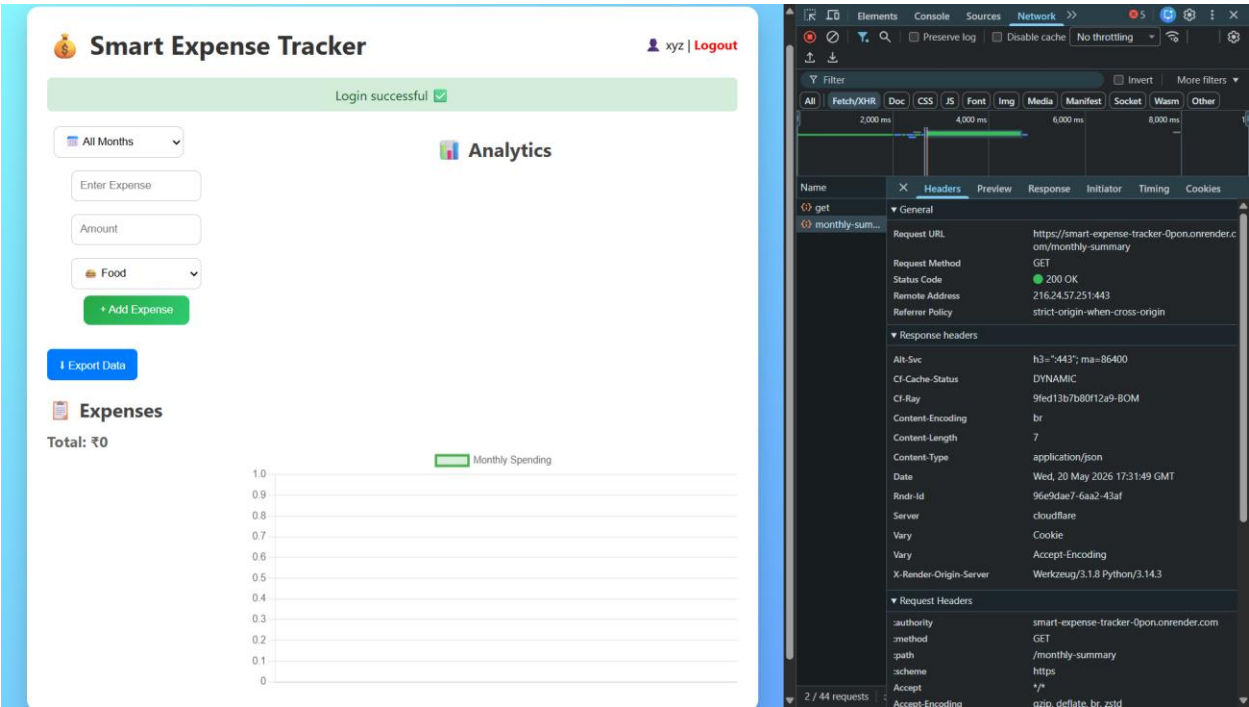


Figure 5: Monthly summary API header inspection.

12. Empty JSON Response Analysis

The monthly-summary endpoint initially returned an empty JSON response because no expense records were available at that time. This demonstrates structured API response handling practices.

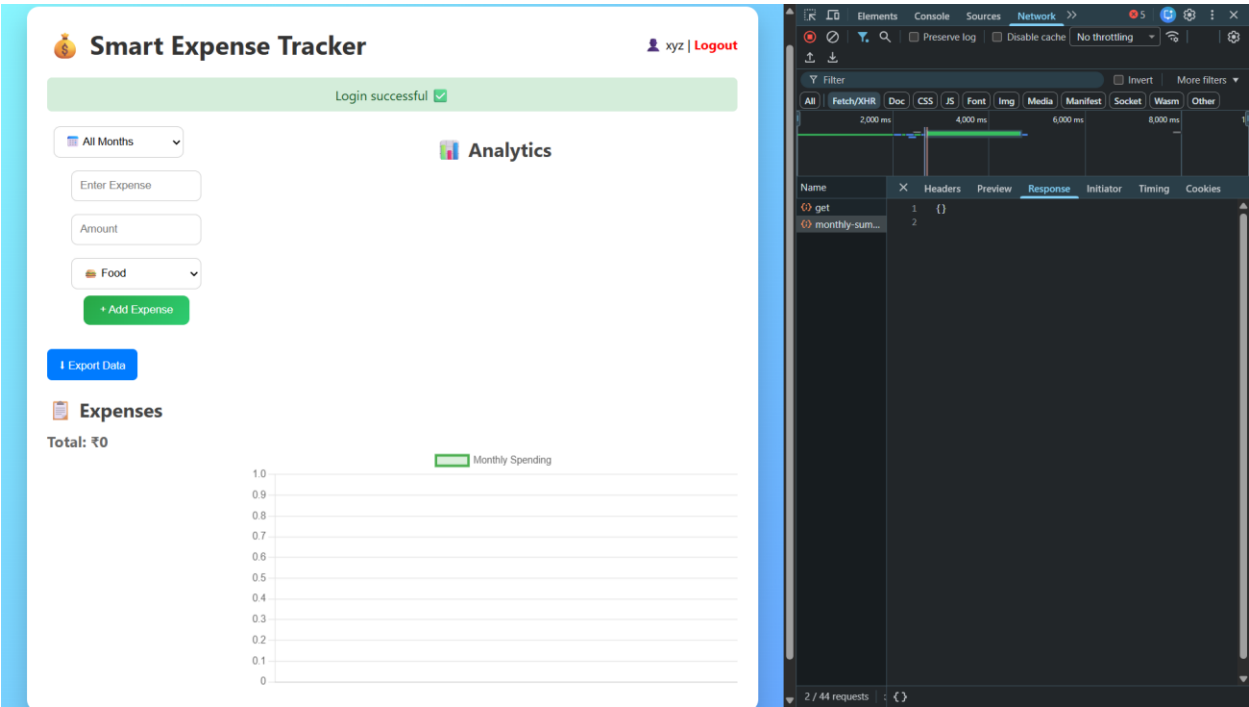


Figure 6: Empty JSON response returned from API endpoint.

13. Add Expense API Payload Analysis

This screenshot demonstrates the structured JSON payload submitted during expense creation. The frontend transmitted expense-related information to the backend using a POST API request.

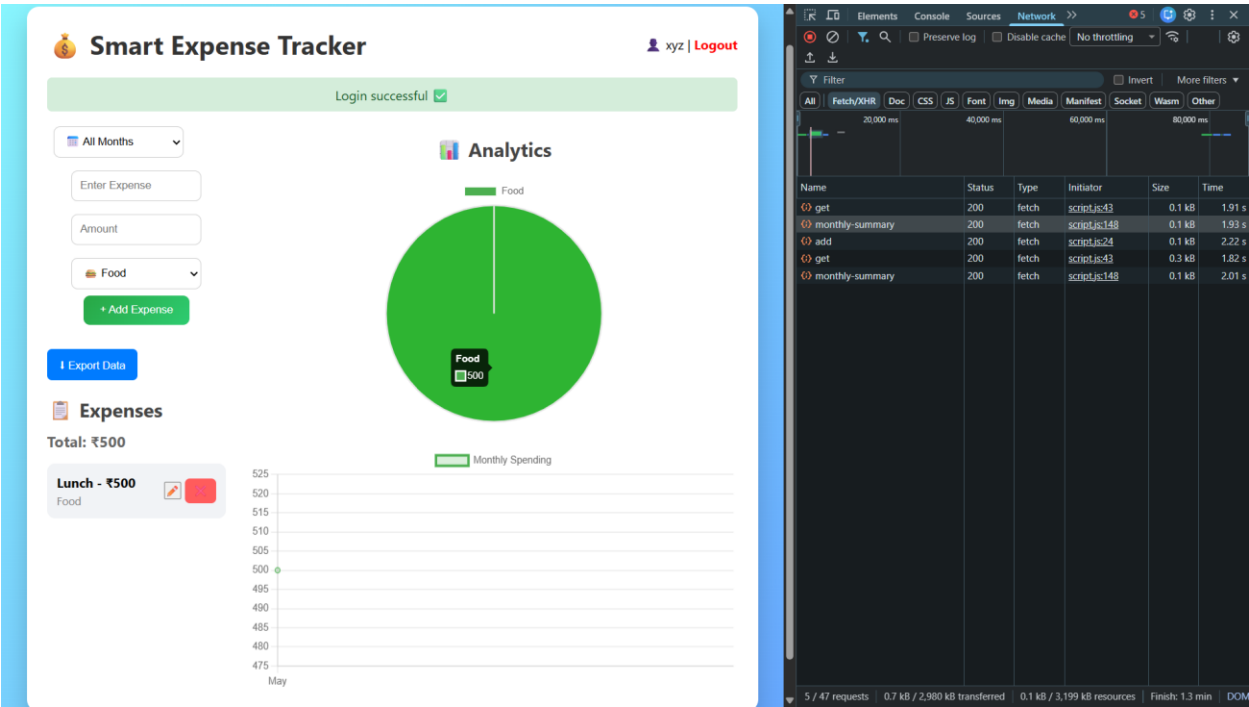


Figure 7: Expense submission payload analysis.

14. Add Expense API Response Analysis

This screenshot displays the JSON response returned by the backend after successful expense submission. The response contained structured expense data required for dashboard updates.

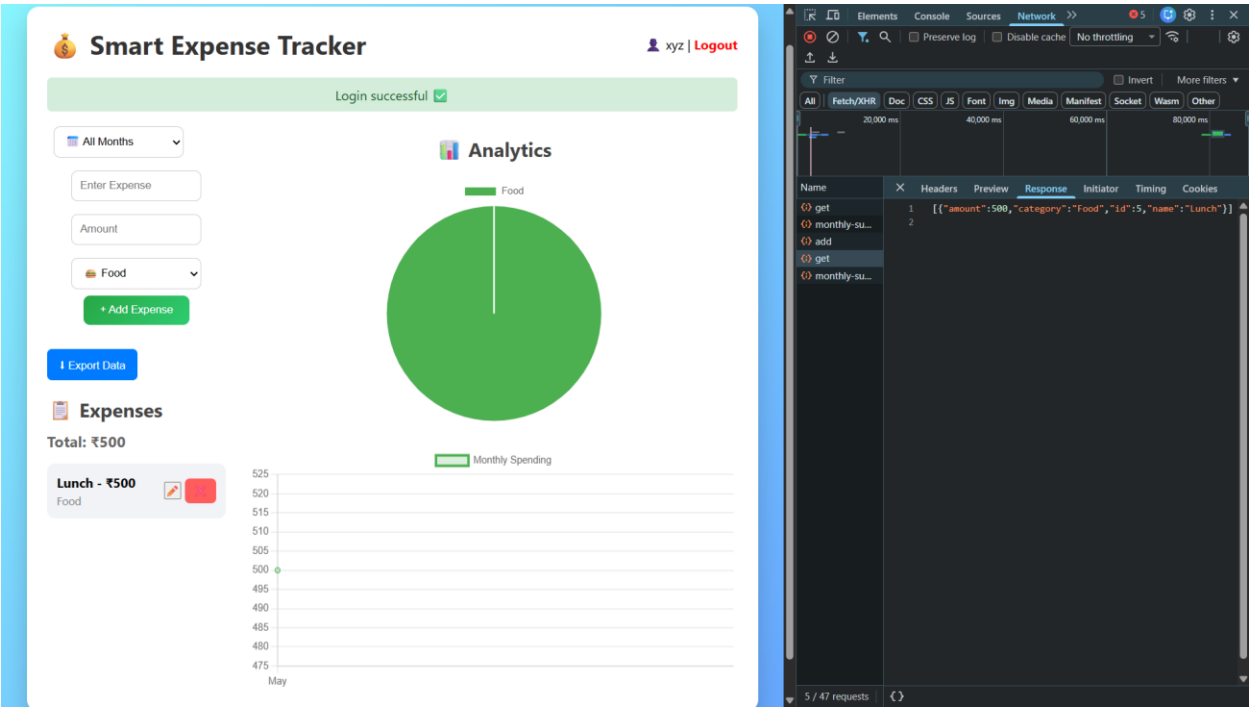


Figure 8: API response received after expense submission.

15. Detailed Payload Structure Analysis

Detailed payload inspection was performed to understand frontend-to-backend data transfer mechanisms. Proper JSON formatting and structured API communication were observed.

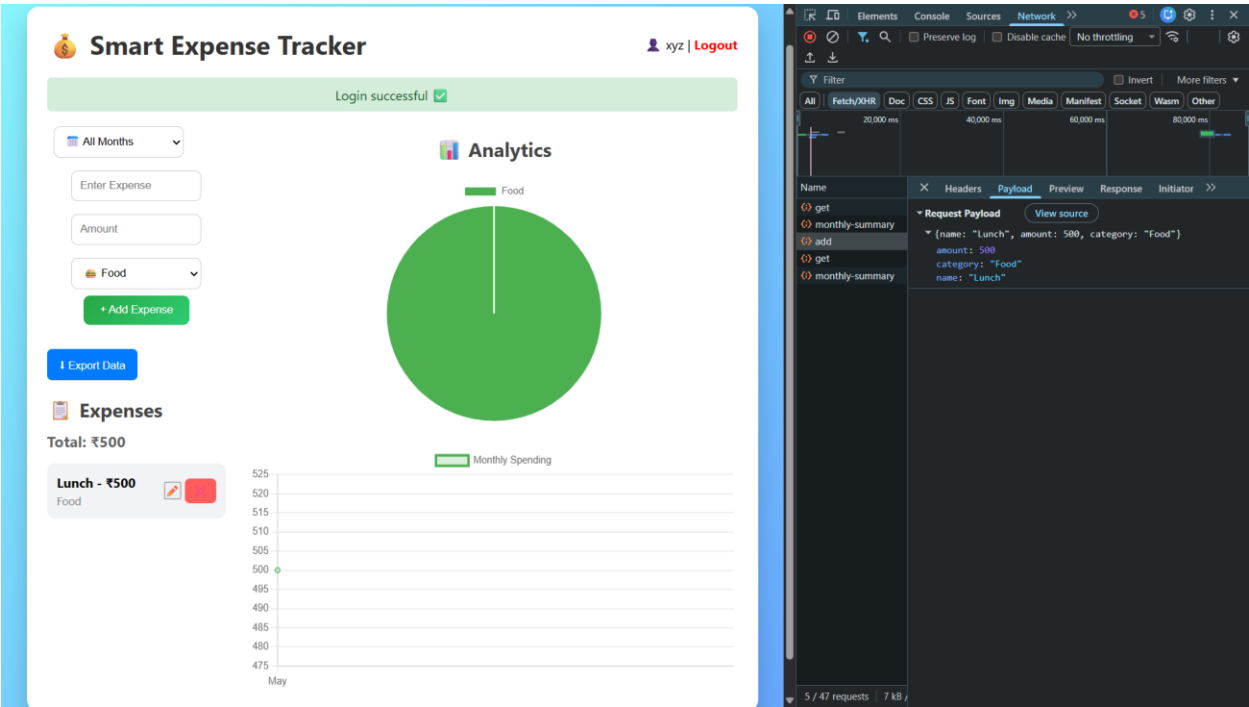


Figure 9: Detailed payload structure inspection.

16. Detailed Request Header & Session Analysis

This screenshot demonstrates detailed request header analysis and session handling mechanisms. Session cookies were visible within authenticated requests, indicating session-based authentication persistence.

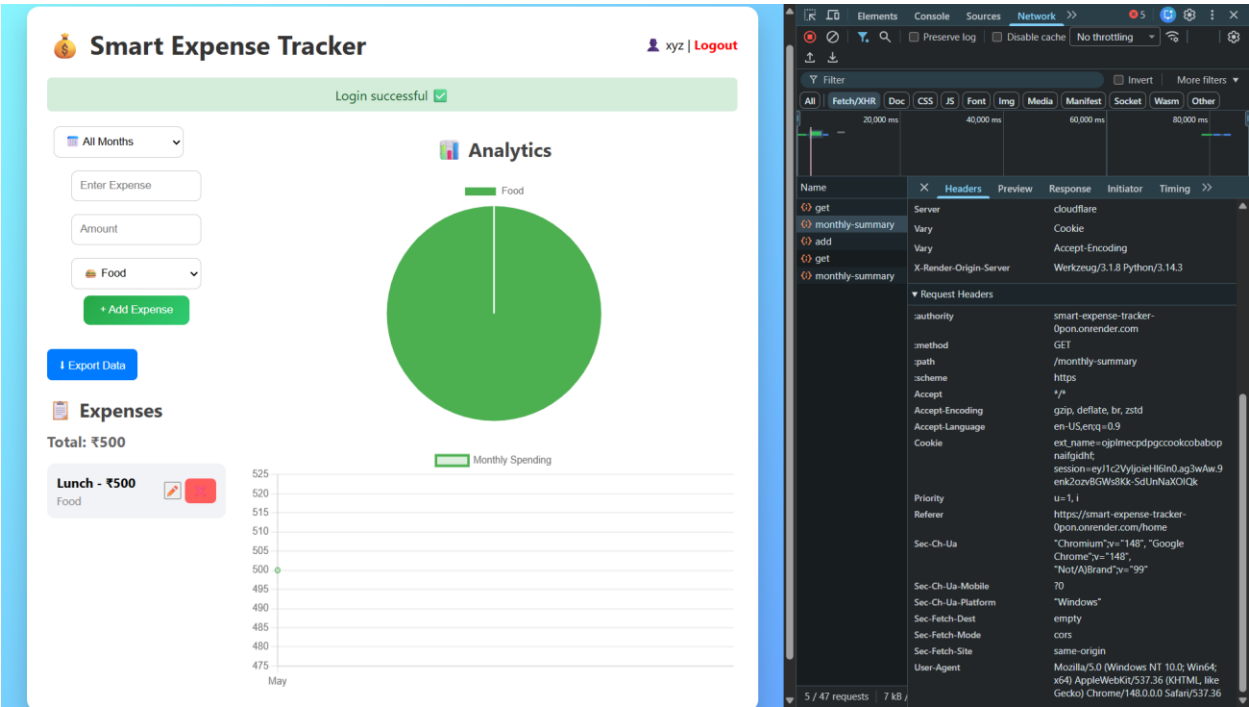


Figure 10: Request header and session cookie analysis.

17. Security Observations

- The assessment identified multiple secure API communication practices within the Smart Expense Tracker application. HTTPS encryption was consistently enabled during request transmission, ensuring secure frontend-backend communication.
- Session-based authentication was observed through request cookies present in authenticated API calls. Structured JSON payloads and responses were used throughout the application workflow.
- Minimal data exposure was identified during response analysis, as no sensitive credentials or confidential information were exposed within visible API responses.
- The application demonstrated organized API communication architecture; however, proper backend validation and authorization controls remain important to prevent risks such as malicious payload manipulation or unauthorized object access.

18. Conclusion

The API analysis successfully demonstrated practical frontend-backend communication workflows within the Smart Expense Tracker application. Using Chrome DevTools, multiple API requests, payload structures, response handling mechanisms, and session management techniques were analyzed.

The assessment provided hands-on exposure to browser-based network monitoring, request-response inspection, and API debugging practices commonly used in modern web application development and security analysis.

Overall, the task significantly improved understanding of API architecture, DevTools debugging workflows, authentication handling, and secure communication practices within full stack web applications.